

Coby White Analysis - FINAL (DATA COLLECTION)

DATA SET-UP

```
In [1]: pip install nba_api pandas
```

```
In [53]: # Import packages
import pandas as pd
import os
import matplotlib as plt
import plotly.express as px
import numpy as np
import time
import random
import requests
from concurrent.futures import ThreadPoolExecutor, as_completed
from scipy.stats import percentileofscore
```

COLLECT DATA

I. General Stats

ALL PLAYERS:

LeagueDashPlayerStats

```
In [ ]: from nba_api.stats.endpoints import leaguedashplayerstats
```

2024-25 Per Game Averages

```
In [ ]: # Get all player stats for 2024-25 season
league_24_25_stats = leaguedashplayerstats.LeagueDashPlayerStats(season='2024-25', per_mode_detailed='PerGame').get_data_frames()[0]
league_24_25_stats
```

```
In [ ]: league_24_25_stats.columns
```

```
In [ ]: # Condense table to relevant columns
league_24_25_general_stats_per_game = league_24_25_stats[['PLAYER_NAME', 'MIN', 'FGM', 'FGA', 'FG_PCT', 'FG3M', 'FG3A', 'FG3_PCT', 'FTM', 'FTA',
'FT_PCT', 'OREB', 'DREB', 'REB', 'AST', 'STL', 'BLK', 'TOV', 'PF',
```

```
    'PTS']].sort_values(by='PTS', ascending=False)
league_24_25_general_stats_per_game
```

```
In [ ]: # Export to CSV
league_24_25_general_stats_per_game.to_csv("league_24_25_general_stats_per_game.csv", index=False)
```

2023-24 Per Game Averages

```
In [ ]: # Get all player stats for 2023-24 season
league_23_24_stats = leaguedashplayerstats.LeagueDashPlayerStats(season='2023-24', per_mode_detailed='PerGame').get_data_frames()[0]
league_23_24_stats
```

```
In [ ]: # Condense table to relevant columns
league_23_24_general_stats_per_game = league_23_24_stats[['PLAYER_NAME', 'MIN', 'FGM', 'FGA', 'FG_PCT', 'FG3M', 'FG3A', 'FG3_PCT', 'FTM', 'FTA',
    'FT_PCT', 'OREB', 'DREB', 'REB', 'AST', 'STL', 'BLK', 'TOV', 'PF',
    'PTS']].sort_values(by='PTS', ascending=False)
league_23_24_general_stats_per_game
```

```
In [ ]: # Export to CSV
league_23_24_general_stats_per_game.to_csv("league_23_24_general_stats_per_game.csv", index=False)
```

Coby White General Stats

```
In [ ]: Coby_White_23_24_stats = league_23_24_stats[league_23_24_stats['PLAYER_NAME'] == 'Coby White']
Coby_White_23_24_stats.columns
```

```
In [ ]: Coby_White_23_24_general_stats_per_game = league_23_24_general_stats_per_game[league_23_24_general_stats_per_game['PLAYER_NAME'] == 'Coby White']
Coby_White_23_24_general_stats_per_game
```

LeagueDashPlayerStats - Advanced Stats

2024-25 Per Game Averages

```
In [ ]: # Get all player stats for 2024-25 season
league_24_25_advanced_stats = leaguedashplayerstats.LeagueDashPlayerStats(
    season='2024-25', per_mode_detailed='PerGame', measure_type_detailed_defense='Advanced'
).get_data_frames()[0]

league_24_25_advanced_stats
```

```
In [ ]: league_24_25_advanced_stats.columns
```

```
In [ ]: # Condense table to relevant columns
league_24_25_advanced_stats_per_game = league_24_25_advanced_stats[['PLAYER_NAME', 'EFG_PCT', 'TS_PCT', 'USG_PCT', 'E_USG_PCT', 'PACE', 'E_PACE', 'PIE'
    ]]
league_24_25_advanced_stats_per_game
```

```
In [ ]: # Export to CSV
league_24_25_advanced_stats_per_game.to_csv("league_24_25_advanced_stats_per_game.csv", index=False)
```

Coby White Advanced Stats

```
In [ ]: Coby_White_23_24_advanced_stats_per_game = league_24_25_advanced_stats_per_game[league_24_25_advanced_stats_per_game['PLAYER_NAME'] == 'Coby White']
Coby_White_23_24_advanced_stats_per_game
```

II. Game Logs

ALL PLAYER LOGS:

```
In [5]: from nba_api.stats.endpoints import PlayerGameLog
from nba_api.stats.static import players
```

```
In [9]: # Get all active player IDs
all_players = players.get_active_players()
player_ids = [p['id'] for p in all_players]
player_dict = {p['id']: p['full_name'] for p in all_players}
```

2024-25 Totals

```
In [9]: # *Took 1 hour+ to run*
# Objective: Loop through game logs to get data for all players in 24-25 season
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 5 retries
def fetch_player_game_logs(player_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call game logs for each player with increased timeout (from default 30 to 60)
            response = PlayerGameLog(
                player_id=player_id,
                season="2024-25",
                timeout=60
            )

            data_frames = response.get_data_frames()

            # Ensure game log data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No game log data for player ID {player_id}")
                return None
```

```

    game_log_df = data_frames[0].copy()
    game_log_df["PLAYER_ID"] = player_id
    game_log_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
    return game_log_df

# Return error message
except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
    print(f"⚠️ Connection issue for player ID {player_id}: {e}")
except Exception as e:
    print(f"⚠️ Unexpected error retrieving data for player ID {player_id}: {e}")

# Exponential backoff with random jitter to avoid API blocking before next attempt
attempt += 1
sleep_time = (2 ** attempt) + random.uniform(0, 1)
print(f"🔄 Retrying player ID {player_id} in {sleep_time:.2f} seconds...")
time.sleep(sleep_time)

print(f"❌ Failed to retrieve data for player ID {player_id} after {retries} attempts.")
return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store game log data for all players in a list
all_players_game_log_24_25 = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_player_game_logs, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to player dictionary
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_game_log_24_25.append(result)

# Combine results into a single DF
if all_players_game_log_24_25:
    all_players_game_log_24_25 = pd.concat(all_players_game_log_24_25, ignore_index=True)

print("✅ Data retrieval complete!")

```

```
In [ ]: all_players_game_log_24_25.columns
```

```
In [ ]: all_players_game_log_24_25.shape[0]
```

```
In [ ]: all_players_game_log_24_25.head()
```

```
In [11]: # Export to CSV
all_players_game_log_24_25.to_csv("all_players_game_log_24_25.csv", index=False)
```

2023-24 Totals

```
In [11]: # *Took 1 hour+ to run*
# Objective: Loop through game logs to get data for all players in 23-24 season
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 5 retries
def fetch_player_game_logs(player_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call game logs for each player with increased timeout (from default 30 to 60)
            response = PlayerGameLog(
                player_id=player_id,
                season="2023-24",
                timeout=60
            )

            data_frames = response.get_data_frames()

            # Ensure game log data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No game log data for player ID {player_id}")
                return None

            game_log_df = data_frames[0].copy()
            game_log_df["PLAYER_ID"] = player_id
            game_log_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
            return game_log_df

        # Return error message
        except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
            print(f"⚠️ Connection issue for player ID {player_id}: {e}")
        except Exception as e:
            print(f"⚠️ Unexpected error retrieving data for player ID {player_id}: {e}")

        # Exponential backoff with random jitter to avoid API blocking before next attempt
        attempt += 1
        sleep_time = (2 ** attempt) + random.uniform(0, 1)
        print(f"🔄 Retrying player ID {player_id} in {sleep_time:.2f} seconds...")
        time.sleep(sleep_time)

    print(f"💥 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1
```

```

# Store game log data for all players in a list
all_players_game_log_23_24 = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_player_game_logs, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to player dictionary
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_game_log_23_24.append(result)

# Combine results into a single DF
if all_players_game_log_23_24:
    all_players_game_log_23_24 = pd.concat(all_players_game_log_23_24, ignore_index=True)

print("✅ Data retrieval complete!")

```

```
In [13]: all_players_game_log_23_24.shape[0]
```

```
In [15]: all_players_game_log_23_24.head()
```

```
In [17]: # Export to CSV
all_players_game_log_23_24.to_csv("all_players_game_log_23_24.csv", index=False)
```

III. Role Stats

USAGE:

LeagueGameLog: *Get a list of all game IDs from every season*

```
In [7]: from nba_api.stats.endpoints import LeagueGameLog
```

2024-25 Game IDs

```
In [9]: league_game_log_24_25 = LeagueGameLog(season='2024-25', season_type_all_star='Regular Season')
league_game_log_24_25_df = league_game_log_24_25.get_data_frames()[0]
league_game_log_24_25_df
```

```
In [11]: league_game_log_24_25_df['GAME_DATE'].dtype
```

```
In [13]: # Convert game_date into datetime object
league_game_log_24_25_df['GAME_DATE'] = pd.to_datetime(league_game_log_24_25_df['GAME_DATE'])
```

```
In [15]: game_ids_24_25 = league_game_log_24_25_df[['GAME_ID', 'GAME_DATE']]
game_ids_24_25
```

```
In [17]: all_star_date = pd.to_datetime("2025-02-14")
```

```
In [19]: game_ids_24_25_1H = game_ids_24_25[game_ids_24_25['GAME_DATE'] < all_star_date]
game_ids_24_25_1H
```

```
In [21]: game_ids_24_25_2H = game_ids_24_25[game_ids_24_25['GAME_DATE'] > all_star_date]
game_ids_24_25_2H
```

2023-24 Game IDs

```
In [25]: league_game_log_23_24 = LeagueGameLog(season='2023-24', season_type_all_star='Regular Season')
league_game_log_23_24_df = league_game_log_23_24.get_data_frames()[0]
league_game_log_23_24_df
```

```
In [27]: # Convert game_date into datetime object
league_game_log_23_24_df['GAME_DATE'] = pd.to_datetime(league_game_log_23_24_df['GAME_DATE'])
```

```
In [29]: game_ids_23_24 = league_game_log_23_24_df[['GAME_ID', 'GAME_DATE']]
game_ids_23_24
```

BoxScoreUsageV2

```
In [114...] from nba_api.stats.endpoints import BoxScoreUsageV2
```

2024-25 Pre All-Star

```
In [124...] # Takes 4+ hours to run
# Function to fetch and process data for each box score with up to 5 retries
def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)
            response = BoxScoreUsageV2(game_id=game_id, timeout=60)
            data_frames = response.get_data_frames()

            # Ensure box score data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No box score data for game ID {game_id}")
                return None

            box_score_df = data_frames[0].copy()
            box_score_df["GAME_ID"] = game_id
```

```

        return box_score_df

    # Return error message
    except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
        print(f"⚠️ Connection issue for game ID {game_id}: {e}")
    except Exception as e:
        print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

    # Exponential backoff with random jitter to avoid API blocking before next attempt
    attempt += 1
    sleep_time = (2 ** attempt) + random.uniform(0, 1)
    print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
    time.sleep(sleep_time)

    print(f"🚫 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_usage_24_25_1H = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_24_25_1H["GAME_ID"]
    }

    # Process results as soon as they're completed and add results from future_to_game function to box score list
    for future in as_completed(future_to_game):
        result = future.result()
        if result is not None:
            box_score_usage_24_25_1H.append(result)

# Combine results into a DF
if box_score_usage_24_25_1H:
    usage_24_25_1H = pd.concat(box_score_usage_24_25_1H, ignore_index=True)

    print("✅ Box Score Usage data retrieval complete!")

```

In [127... `usage_24_25_1H.columns`

In [129... `usage_24_25_1H.head()`

In [131... `usage_24_25_1H.shape[0]`

In [133... `# Export to CSV`
`usage_24_25_1H.to_csv("usage_24_25_1H.csv", index=False)`

2024-25 Post All-Star

```
In [4]: # Takes 4+ hours to run
# Function to fetch and process data for each box score with up to 5 retries
def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)
            response = BoxScoreUsageV2(game_id=game_id, timeout=60)
            data_frames = response.get_data_frames()

            # Ensure box score data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No box score data for game ID {game_id}")
                return None

            box_score_df = data_frames[0].copy()
            box_score_df["GAME_ID"] = game_id
            return box_score_df

        # Return error message
        except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
            print(f"⚠️ Connection issue for game ID {game_id}: {e}")
        except Exception as e:
            print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

        # Exponential backoff with random jitter to avoid API blocking before next attempt
        attempt += 1
        sleep_time = (2 ** attempt) + random.uniform(0, 1)
        print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
        time.sleep(sleep_time)

    print(f"💥 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_usage_24_25_2H = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_24_25_2H["GAME_ID"]
    }

# Process results as soon as they're completed and add results from future_to_game function to box score list
```

```

    for future in as_completed(future_to_game):
        result = future.result()
        if result is not None:
            box_score_usage_24_25_2H.append(result)

# Combine results into a DF
if box_score_usage_24_25_2H:
    usage_24_25_2H = pd.concat(box_score_usage_24_25_2H, ignore_index=True)

    print("✅ Box Score Usage data retrieval complete!")

```

In [137... `usage_24_25_2H.shape[0]`

In [139... *# Export to CSV*
`usage_24_25_2H.to_csv("usage_24_25_2H.csv", index=False)`

2023-24

In [141... *# Takes 4+ hours to run*
Function to fetch and process data for each box score with up to 5 retries

```

def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)
            response = BoxScoreUsageV2(game_id=game_id, timeout=60)
            data_frames = response.get_data_frames()

            # Ensure box score data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No box score data for game ID {game_id}")
                return None

            box_score_df = data_frames[0].copy()
            box_score_df["GAME_ID"] = game_id
            return box_score_df

        # Return error message
        except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
            print(f"⚠️ Connection issue for game ID {game_id}: {e}")
        except Exception as e:
            print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

        # Exponential backoff with random jitter to avoid API blocking before next attempt
        attempt += 1
        sleep_time = (2 ** attempt) + random.uniform(0, 1)
        print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
        time.sleep(sleep_time)

```

```

    print(f"🚨 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_usage_23_24 = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_23_24["GAME_ID"]
    }

    # Process results as soon as they're completed and add results from future_to_game function to box score list
    for future in as_completed(future_to_game):
        result = future.result()
        if result is not None:
            box_score_usage_23_24.append(result)

# Combine results into a DF
if box_score_usage_23_24:
    usage_23_24 = pd.concat(box_score_usage_23_24, ignore_index=True)

    print("✅ Box Score Usage data retrieval complete!")

```

```
In [47]: usage_23_24.shape[0]
```

```
In [145... # Export to CSV
usage_23_24.to_csv("usage_23_24.csv", index=False)
```

AGGRESSION

BoxScoreMiscV2

```
In [31]: from nba_api.stats.endpoints import BoxScoreMiscV2
```

2024-25 Pre All-Star

```
In [31]: # Takes ~6 hours to run
# Function to fetch and process data for each box score with up to 5 retries
def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)

```

```

response = BoxScoreMiscV2(game_id=game_id, timeout=60)
data_frames = response.get_data_frames()

# Ensure box score data exists and return data frame per player or stop retrying if no valid data
if not data_frames or data_frames[0].empty:
    print(f"❌ No box score data for game ID {game_id}")
    return None

box_score_df = data_frames[0].copy()
box_score_df["GAME_ID"] = game_id
return box_score_df

# Return error message
except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
    print(f"⚠️ Connection issue for game ID {game_id}: {e}")
except Exception as e:
    print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

# Exponential backoff with random jitter to avoid API blocking before next attempt
attempt += 1
sleep_time = (2 ** attempt) + random.uniform(0, 1)
print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
time.sleep(sleep_time)

print(f"💢 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_misc_24_25_1H = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_24_25_1H["GAME_ID"]
    }

# Process results as soon as they're completed and add results from future_to_game function to box score list
for future in as_completed(future_to_game):
    result = future.result()
    if result is not None:
        box_score_misc_24_25_1H.append(result)

# Combine results into a DF
if box_score_misc_24_25_1H:
    misc_24_25_1H = pd.concat(box_score_misc_24_25_1H, ignore_index=True)

print(f"✅ Box Score Miscellaneous data retrieval complete!")

```

```
In [49]: misc_24_25_1H.shape[0]
```

```
In [51]: misc_24_25_1H.columns
```

```
In [55]: # Combine List of dataframes into a single DF

box_score_misc_24_25_1H_separate_DFs = box_score_misc_24_25_1H.copy()
box_score_misc_24_25_1H = pd.concat(box_score_misc_24_25_1H_separate_DFs, ignore_index=True)
```

```
In [57]: # Export to CSV
box_score_misc_24_25_1H.to_csv("box_score_misc_24_25_1H.csv", index=False)
```

2024-25 Post All-Star

```
In [ ]: # Takes ~6 hours to run
# Function to fetch and process data for each box score with up to 5 retries
def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)
            response = BoxScoreMiscV2(game_id=game_id, timeout=60)
            data_frames = response.get_data_frames()

            # Ensure box score data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:
                print(f"❌ No box score data for game ID {game_id}")
                return None

            box_score_df = data_frames[0].copy()
            box_score_df["GAME_ID"] = game_id
            return box_score_df

        # Return error message
        except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
            print(f"⚠️ Connection issue for game ID {game_id}: {e}")
        except Exception as e:
            print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

        # Exponential backoff with random jitter to avoid API blocking before next attempt
        attempt += 1
        sleep_time = (2 ** attempt) + random.uniform(0, 1)
        print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
        time.sleep(sleep_time)

    print(f"🚫 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
    return None
```

```

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_misc_24_25_2H = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_24_25_2H["GAME_ID"]
    }

    # Process results as soon as they're completed and add results from future_to_game function to box score list
    for future in as_completed(future_to_game):
        result = future.result()
        if result is not None:
            box_score_misc_24_25_2H.append(result)

# Combine results into a DF
if box_score_misc_24_25_2H:
    misc_24_25_2H = pd.concat(box_score_misc_24_25_2H, ignore_index=True)

    print("✅ Box Score Miscellaneous data retrieval complete!")

```

In [62]: `misc_24_25_2H.shape[0]`

In [64]: `# Combine List of dataframes into a single DF`

```

box_score_misc_24_25_2H_separate_DFs = box_score_misc_24_25_2H.copy()
box_score_misc_24_25_2H = pd.concat(box_score_misc_24_25_2H_separate_DFs, ignore_index=True)

```

In [66]: `# Export to CSV`

```

box_score_misc_24_25_2H.to_csv("box_score_misc_24_25_2H.csv", index=False)

```

2023-24

In [33]: `# Takes ~6 hours to run`

`# Function to fetch and process data for each box score with up to 5 retries`

```

def fetch_box_scores(game_id, retries=5):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving box scores for game ID {game_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call box scores for each game with increased timeout (from default 30 to 60)
            response = BoxScoreMiscV2(game_id=game_id, timeout=60)
            data_frames = response.get_data_frames()

            # Ensure box score data exists and return data frame per player or stop retrying if no valid data
            if not data_frames or data_frames[0].empty:

```

```

        print(f"❌ No box score data for game ID {game_id}")
        return None

    box_score_df = data_frames[0].copy()
    box_score_df["GAME_ID"] = game_id
    return box_score_df

    # Return error message
    except (requests.exceptions.ConnectionError, requests.exceptions.ReadTimeout) as e:
        print(f"⚠️ Connection issue for game ID {game_id}: {e}")
    except Exception as e:
        print(f"⚠️ Unexpected error retrieving data for game ID {game_id}: {e}")

    # Exponential backoff with random jitter to avoid API blocking before next attempt
    attempt += 1
    sleep_time = (2 ** attempt) + random.uniform(0, 1)
    print(f"🔄 Retrying game ID {game_id} in {sleep_time:.2f} seconds...")
    time.sleep(sleep_time)

    print(f"💩 Failed to retrieve data for game ID {game_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 1

# Store box score data for all games in a list
box_score_misc_23_24 = []

# Run multiple requests in parallel and submit future objects into game dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_game = {
        executor.submit(fetch_box_scores, gid): gid
        for gid in game_ids_23_24["GAME_ID"]
    }

    # Process results as soon as they're completed and add results from future_to_game function to box score list
    for future in as_completed(future_to_game):
        result = future.result()
        if result is not None:
            box_score_misc_23_24.append(result)

# Combine results into a DF
if box_score_misc_23_24:
    misc_23_24 = pd.concat(box_score_misc_23_24, ignore_index=True)

    print("✅ Box Score Miscellaneous data retrieval complete!")

```

In [45]: # Combine List of dataframes into a single DF

```

box_score_misc_23_24_separate_DFs = box_score_misc_23_24.copy()
box_score_misc_23_24 = pd.concat(box_score_misc_23_24_separate_DFs, ignore_index=True)

```

```
In [47]: # Export to CSV
box_score_misc_23_24.to_csv("box_score_misc_23_24.csv", index=False)
```

```
In [51]: box_score_misc_23_24.shape[0]
```

IV. Shot Profile

SHOT LOCATION:

```
In [19]: from nba_api.stats.endpoints import PlayerDashboardByShootingSplits
from nba_api.stats.static import players
```

```
In [21]: # Get all active player IDs
all_players = players.get_active_players()
player_ids = [p['id'] for p in all_players]
player_dict = {p['id']: p['full_name'] for p in all_players}
```

2024-25 Pre All-Star Totals

```
In [23]: # Loop through shooting splits to get shot location data for all players in 24-25 season pre all-star
# Takes ~10 minutes to run
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 3 retries
def fetch_shooting_splits(player_id, retries=3):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call shooting splits for each player
            response = PlayerDashboardByShootingSplits(
                player_id=player_id,
                season="2024-25",
                season_segment_nullable="Pre All-Star"
            )

            data_frames = response.get_data_frames()

            # Ensure shot location data (data frame 3) exists and return data frame per player
            if len(data_frames) > 3 and not data_frames[3].empty:
                shooting_splits_df = data_frames[3].copy()
                shooting_splits_df["PLAYER_ID"] = player_id
                shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
                return shooting_splits_df
```



```

        # Stop retrying if no valid data
        print(f"❌ No shot location data for player ID {player_id}")
        return None

    # Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt
    except Exception as e:
        print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
        attempt += 1
        time.sleep(2 ** attempt + random.uniform(0, 1))

    # Give up after 3 retries
    print(f"🛑 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store shot location data for all players in a list
all_players_shot_location_24_25_1H = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to shot location list
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_shot_location_24_25_1H.append(result)

# Combine results into a single DF
if all_players_shot_location_24_25_1H:
    all_players_shot_location_24_25_1H = pd.concat(all_players_shot_location_24_25_1H, ignore_index=True)

print(f"✅ Data retrieval complete!")

```

```
In [40]: all_players_shot_location_24_25_1H.head()
```

```
In [38]: all_players_shot_location_24_25_1H.shape[0]
```

```
In [42]: # Export to CSV
all_players_shot_location_24_25_1H.to_csv("all_players_shot_location_24_25_1H.csv", index=False)
```

2024-25 Post All-Star Totals

```
In [44]: # Loop through shooting splits to get shot location data for all players in 24-25 season post all-star
# Takes ~7 minutes to run
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}
```

```

# Function to fetch and process data for a single player with up to 3 retries
def fetch_shooting_splits(player_id, retries=3):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call shooting splits for each player
            response = PlayerDashboardByShootingSplits(
                player_id=player_id,
                season="2024-25",
                season_segment_nullable="Post All-Star"
            )

            data_frames = response.get_data_frames()

            # Ensure shot location data (data frame 3) exists and return data frame per player
            if len(data_frames) > 3 and not data_frames[3].empty:
                shooting_splits_df = data_frames[3].copy()
                shooting_splits_df["PLAYER_ID"] = player_id
                shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
                return shooting_splits_df

            # Stop retrying if no valid data
            print(f"❌ No shot location data for player ID {player_id}")
            return None

        # Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt
        except Exception as e:
            print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
            attempt += 1
            time.sleep(2 ** attempt + random.uniform(0, 1))

    # Give up after 3 retries
    print(f"💩 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store shot location data for all players in a list
all_players_shot_location_24_25_2H = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

# Process results as soon as they're completed and add results from future_to_player function to shot location list
for future in as_completed(future_to_player):
    result = future.result()
    if result is not None:
        all_players_shot_location_24_25_2H.append(result)

```

```
# Combine results into a single DF
if all_players_shot_location_24_25_2H:
    all_players_shot_location_24_25_2H = pd.concat(all_players_shot_location_24_25_2H, ignore_index=True)

print("✅ Data retrieval complete!")
```

```
In [47]: all_players_shot_location_24_25_2H.shape[0]
```

```
In [ ]: all_players_shot_location_24_25_2H.head()
```

```
In [49]: # Export to CSV
all_players_shot_location_24_25_2H.to_csv("all_players_shot_location_24_25_2H.csv", index=False)
```

2023-24 Totals

```
In [51]: # Loop through shooting splits to get shot location data for all players in 23-24 season
# Takes 7 minutes to run
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 3 retries
def fetch_shooting_splits(player_id, retries=3):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call shooting splits for each player
            response = PlayerDashboardByShootingSplits(
                player_id=player_id,
                season="2023-24"
            )

            data_frames = response.get_data_frames()

            # Ensure shot location data (data frame 3) exists and return data frame per player
            if len(data_frames) > 3 and not data_frames[3].empty:
                shooting_splits_df = data_frames[3].copy()
                shooting_splits_df["PLAYER_ID"] = player_id
                shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
                return shooting_splits_df

            # Stop retrying if no valid data
            print(f"❌ No shot location data for player ID {player_id}")
            return None

        # Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt
        except Exception as e:
            print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
```

```

        attempt += 1
        time.sleep(2 ** attempt + random.uniform(0, 1))

    # Give up after 3 retries
    print(f"🚨 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store shot location data for all players in a List
all_players_shot_location_23_24 = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to shot location list
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_shot_location_23_24.append(result)

# Combine results into a single DF
if all_players_shot_location_23_24:
    all_players_shot_location_23_24 = pd.concat(all_players_shot_location_23_24, ignore_index=True)

print("✅ Data retrieval complete!")

```

```
In [53]: all_players_shot_location_23_24.shape[0]
```

```
In [ ]: all_players_shot_location_23_24.head()
```

```
In [55]: # Export to CSV
all_players_shot_location_23_24.to_csv("all_players_shot_location_23_24.csv", index=False)
```

UNASSISTED SHOTS

```
In [55]: from nba_api.stats.endpoints import PlayerDashboardByShootingSplits
from nba_api.stats.static import players
```

```
In [57]: # Get all active player IDs
all_players = players.get_active_players()
player_ids = [p['id'] for p in all_players]
player_dict = {p['id']: p['full_name'] for p in all_players}
```

Test Coby White Stats

```
In [78]: Coby_White_unassisted_shots_24_25 = PlayerDashboardByShootingSplits(  
        player_id=1629632,  
        season="2024-25"  
    ).get_data_frames()[4]  
Coby_White_unassisted_shots_24_25
```

```
In [72]: Coby_White_unassisted_shots_24_25_1H = PlayerDashboardByShootingSplits(  
        player_id=1629632,  
        season="2024-25",  
        season_segment_nullable="Pre All-Star"  
    ).get_data_frames()[4]  
Coby_White_unassisted_shots_24_25_1H
```

2024-25 Pre All-Star Totals

```
In [61]: # Loop through shooting splits to get unassisted shots data for all players in 24-25 season pre all-star  
# Takes ~1.75 hours to run  
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors  
  
player_info = {player["id"]: player["full_name"] for player in all_players}  
  
# Function to fetch and process data for a single player with up to 3 retries  
def fetch_shooting_splits(player_id, retries=3):  
    attempt = 0  
    while attempt < retries:  
        try:  
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")  
  
            # Call shooting splits for each player  
            response = PlayerDashboardByShootingSplits(  
                player_id=player_id,  
                season="2024-25",  
                season_segment_nullable="Pre All-Star"  
            )  
  
            data_frames = response.get_data_frames()  
  
            # Ensure unassisted shots data (data frame 4) exists and return data frame per player  
            if len(data_frames) > 3 and not data_frames[4].empty:  
                shooting_splits_df = data_frames[4].copy()  
                shooting_splits_df["PLAYER_ID"] = player_id  
                shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")  
                return shooting_splits_df  
  
            # Stop retrying if no valid data  
            print(f"❌ No unassisted shots data for player ID {player_id}")  
            return None  
  
        # Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt  
    except Exception as e:  
        print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
```

```

        attempt += 1
        time.sleep(2 ** attempt + random.uniform(0, 1))

    # Give up after 3 retries
    print(f"🚨 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store unassisted shots data for all players in a List
all_players_unassisted_shots_24_25_1H = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to unassisted shots data List
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_unassisted_shots_24_25_1H.append(result)

# Combine results into a single DF
if all_players_unassisted_shots_24_25_1H:
    all_players_unassisted_shots_24_25_1H = pd.concat(all_players_unassisted_shots_24_25_1H, ignore_index=True)

print("✅ Data retrieval complete!")

```

```
In [65]: all_players_unassisted_shots_24_25_1H.head()
```

```
In [67]: all_players_unassisted_shots_24_25_1H.shape[0]
```

```
In [69]: # Export to CSV
all_players_unassisted_shots_24_25_1H.to_csv("all_players_unassisted_shots_24_25_1H.csv", index=False)
```

2024-25 Post All-Star Totals

```
In [72]: # Loop through shooting splits to get unassisted shots data for all players in 24-25 season post all-star
# Takes ~1.75 hours to run
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 3 retries
def fetch_shooting_splits(player_id, retries=3):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

```

```

# Call shooting splits for each player
response = PlayerDashboardByShootingSplits(
    player_id=player_id,
    season="2024-25",
    season_segment_nullable="Post All-Star"
)

data_frames = response.get_data_frames()

# Ensure unassisted shots data (data frame 4) exists and return data frame per player
if len(data_frames) > 3 and not data_frames[4].empty:
    shooting_splits_df = data_frames[4].copy()
    shooting_splits_df["PLAYER_ID"] = player_id
    shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
    return shooting_splits_df

# Stop retrying if no valid data
print(f"❌ No unassisted shots data for player ID {player_id}")
return None

# Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt
except Exception as e:
    print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
    attempt += 1
    time.sleep(2 ** attempt + random.uniform(0, 1))

# Give up after 3 retries
print(f"🛑 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
return None

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store unassisted shots data for all players in a List
all_players_unassisted_shots_24_25_2H = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

# Process results as soon as they're completed and add results from future_to_player function to unassisted shots data List
for future in as_completed(future_to_player):
    result = future.result()
    if result is not None:
        all_players_unassisted_shots_24_25_2H.append(result)

# Combine results into a single DF
if all_players_unassisted_shots_24_25_2H:
    all_players_unassisted_shots_24_25_2H = pd.concat(all_players_unassisted_shots_24_25_2H, ignore_index=True)

print(f"✅ Data retrieval complete!")

```

```
In [74]: all_players_unassisted_shots_24_25_2H.shape[0]
```

```
In [76]: all_players_unassisted_shots_24_25_2H.head()
```

```
In [78]: # Export to CSV
all_players_unassisted_shots_24_25_2H.to_csv("all_players_unassisted_shots_24_25_2H.csv", index=False)
```

2023-24 Totals

```
In [80]: # Loop through shooting splits to get unassisted shots data for all players in 23-24 season
# Takes ~30 min to run
# Method: Loop through each player_id using parallel threading, smart rate-limiting, and a retry mechanism to avoid NBA rate limit errors

player_info = {player["id"]: player["full_name"] for player in all_players}

# Function to fetch and process data for a single player with up to 3 retries
def fetch_shooting_splits(player_id, retries=3):
    attempt = 0
    while attempt < retries:
        try:
            print(f"Retrieving data for player ID {player_id} (Attempt {attempt+1}) at {time.time()}...")

            # Call shooting splits for each player
            response = PlayerDashboardByShootingSplits(
                player_id=player_id,
                season="2023-24"
            )

            data_frames = response.get_data_frames()

            # Ensure unassisted shots data (data frame 4) exists and return data frame per player
            if len(data_frames) > 3 and not data_frames[4].empty:
                shooting_splits_df = data_frames[4].copy()
                shooting_splits_df["PLAYER_ID"] = player_id
                shooting_splits_df["FULL_NAME"] = player_info.get(player_id, "Unknown")
                return shooting_splits_df

            # Stop retrying if no valid data
            print(f"❌ No unassisted shots data for player ID {player_id}")
            return None

        # Return error message and add exponential backoff + jitter (time.sleep) to avoid API blocking before next attempt
        except Exception as e:
            print(f"⚠️ Error retrieving data for player ID {player_id}: {e}")
            attempt += 1
            time.sleep(2 ** attempt + random.uniform(0, 1))

    # Give up after 3 retries
    print(f"🛑 Failed to retrieve data for player ID {player_id} after {retries} attempts.")
    return None
```



```

# Limit concurrency to reduce rate-limit issues
max_workers = 3

# Store unassisted shots data for all players in a List
all_players_unassisted_shots_23_24 = []

# Run multiple requests in parallel and submit future objects into player dictionary
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    future_to_player = {executor.submit(fetch_shooting_splits, pid): pid for pid in player_ids}

    # Process results as soon as they're completed and add results from future_to_player function to unassisted shots data List
    for future in as_completed(future_to_player):
        result = future.result()
        if result is not None:
            all_players_unassisted_shots_23_24.append(result)

# Combine results into a single DF
if all_players_unassisted_shots_23_24:
    all_players_unassisted_shots_23_24 = pd.concat(all_players_unassisted_shots_23_24, ignore_index=True)

print("✅ Data retrieval complete!")

```

```
In [84]: all_players_unassisted_shots_23_24.head()
```

```
In [86]: all_players_unassisted_shots_23_24.shape[0]
```

```
In [88]: # Export to CSV
all_players_unassisted_shots_23_24.to_csv("all_players_unassisted_shots_23_24.csv", index=False)
```

```
In [ ]: league_24_25_general_stats_per_game = pd.read_csv("C:\\Users\\mmillan\\OneDrive - United Center\\Coby White Analysis\\league_24_25_general_stats_per_game.csv")
league_24_25_general_stats_per_game
```

```
In [ ]: # Totals for 2024-25 Pre All-Star

# Merge using an outer join to catch all player data and see where some data may be missing
shot_location_totals_24_25_1H = pd.merge(shot_location_totals_24_25_1H, player_game_counts_24_25_1H, on = 'FULL_NAME', how = 'outer')
shot_location_totals_24_25_1H
```

```
In [ ]: # Totals for 2024-25 Post All-Star
shot_location_totals_24_25_2H = pd.merge(shot_location_totals_24_25_2H, player_game_counts_24_25_2H, on = 'FULL_NAME', how = 'outer')
shot_location_totals_24_25_2H
```

```
In [ ]: # Totals for 2023-24
shot_location_totals_23_24 = pd.merge(shot_location_totals_23_24, player_game_counts_23_24, on = 'FULL_NAME', how = 'outer')
shot_location_totals_23_24
```

```
In [ ]: Coby_White_shot_location_totals_23_24 = shot_location_totals_23_24[shot_location_totals_23_24["FULL_NAME"] == "Coby White"]
Coby_White_shot_location_totals_23_24.T
```

```
In [ ]: shot_location_totals_23_24.columns
```

```
In [ ]: # Specify stats that need to be averaged
stat_columns = ['FGM', 'FGA', 'FG3M', 'FG3A', 'BLKA']
```

2024-25 Pre All-Star:

```
In [ ]: shot_location_avg_24_25_1H = shot_location_totals_24_25_1H.copy().reset_index(drop=True)

for col in stat_columns:
    shot_location_avg_24_25_1H[col] = shot_location_avg_24_25_1H[col]/shot_location_avg_24_25_1H["num_games_24_25_1H"]

shot_location_avg_24_25_1H
```

```
In [ ]: # Drop rows with null values
shot_location_avg_24_25_1H_clean = shot_location_avg_24_25_1H.dropna(subset=stat_columns)
shot_location_avg_24_25_1H_clean
```

2024-25 Post All-Star:

```
In [ ]: shot_location_avg_24_25_2H = shot_location_totals_24_25_2H.copy().reset_index(drop=True)

for col in stat_columns:
    shot_location_avg_24_25_2H[col] = shot_location_avg_24_25_2H[col]/shot_location_avg_24_25_2H["num_games_24_25_2H"]

shot_location_avg_24_25_2H
```

```
In [ ]: # Drop rows with null values
shot_location_avg_24_25_2H_clean = shot_location_avg_24_25_2H.dropna(subset=stat_columns)
shot_location_avg_24_25_2H_clean
```

2023-24:

```
In [ ]: shot_location_avg_23_24 = shot_location_totals_23_24.copy().reset_index(drop=True)

for col in stat_columns:
    shot_location_avg_23_24[col] = shot_location_avg_23_24[col]/shot_location_avg_23_24['num_games_23_24']

shot_location_avg_23_24
```

```
In [ ]: # Drop rows with null values
shot_location_avg_23_24_clean = shot_location_avg_23_24.dropna(subset=stat_columns)
shot_location_avg_23_24_clean
```

Test 23-24 avgs

```
In [ ]: shot_location_avg_23_24 = shot_location_totals_23_24.copy().reset_index(drop=True)
shot_location_avg_23_24[shot_location_avg_23_24['FULL_NAME'] == 'Coby White']
```

```
In [ ]: for col in stat_columns:
        shot_location_avg_23_24[col] = shot_location_avg_23_24[col]/shot_location_avg_23_24['num_games_23_24']

        shot_location_avg_23_24
```

```
In [ ]: shot_location_avg_23_24[shot_location_avg_23_24['FULL_NAME'] == 'Coby White']
```

Troubleshoot 23-24 error

```
In [ ]: print(stat_columns)
```

```
In [ ]: print(shot_location_avg_24_25_1H["num_games_24_25_1H"].dtype)
        print(shot_location_avg_24_25_1H["num_games_24_25_1H"].isnull().sum())
        print((shot_location_avg_24_25_1H["num_games_24_25_1H"] == 0).sum())
```

```
In [ ]: print(shot_location_avg_23_24_clean["num_games_23_24"].dtype)
        print(shot_location_avg_23_24_clean["num_games_23_24"].isnull().sum())
        print((shot_location_avg_23_24_clean["num_games_23_24"] == 0).sum())
```

```
In [ ]: shot_location_avg_23_24[stat_columns] = shot_location_avg_23_24[stat_columns].div(
        shot_location_avg_23_24["num_games_23_24"], axis=0
        )
```

```
In [ ]: shot_location_avg_23_24 = shot_location_totals_23_24.copy().reset_index(drop=True)
```

Test Coby averages

```
In [ ]: shot_location_avg_23_24[shot_location_avg_23_24['FULL_NAME'] == 'Coby White']
```

```
In [ ]: test_totals = Coby_White_shot_location_totals_23_24[['FULL_NAME', 'GROUP_VALUE', 'FGM']]
        test_totals
```

```
In [ ]: Coby_White_game_counts
```

```
In [ ]: test_totals_merged = pd.merge(test_totals, Coby_White_game_counts, on = 'FULL_NAME', how = 'outer')
        test_totals_merged
```

```
In [ ]: test_stat_columns = ['FGM']

        test_avg = test_totals_merged.copy()

        for col in test_stat_columns:
            test_avg[col] = test_avg[col]/test_avg['num_games_23_24']

        test_avg
```

```
In [ ]: # ALL player code (for reference)
        shot_location_avg_23_24 = shot_location_totals_23_24.copy()
```

```

for col in stat_columns:
    shot_location_avg_23_24[col] = shot_location_avg_23_24[col]/shot_location_avg_23_24["num_games_23_24"]

shot_location_avg_23_24

```

```

In [ ]: Coby_White_shot_location_avg_24_25_1H = shot_location_avg_24_25_1H_clean[shot_location_avg_24_25_1H_clean["FULL_NAME"] == "Coby White"]
Coby_White_shot_location_avg_24_25_1H.T

```

```

In [ ]: Coby_White_shot_location_avg_24_25_2H = shot_location_avg_24_25_2H_clean[shot_location_avg_24_25_2H_clean["FULL_NAME"] == "Coby White"]
Coby_White_shot_location_avg_24_25_2H.T

```

```

In [ ]: Coby_White_shot_location_avg_23_24 = shot_location_avg_23_24_clean[shot_location_avg_23_24_clean["FULL_NAME"] == "Coby White"]
Coby_White_shot_location_avg_23_24.T

```

```

In [ ]: # Specify stats that need to find percentile ranks for:
stat_columns_percentiles = ['FGM', 'FGA', 'FG3M', 'FG3A', 'BLKA', 'FG_PCT', 'FG3_PCT', 'EFG_PCT',
                             'PCT_AST_2PM', 'PCT_UAST_2PM', 'PCT_AST_3PM', 'PCT_UAST_3PM', 'PCT_AST_FGM', 'PCT_UAST_FGM']

```

```

In [ ]: # Function to calculate percentrank for Coby's stats against all other players in 2024-25 season pre all-star
def get_percentile(value, series):
    return percentileofscore(series, value, kind='rank')

Coby_White_shot_location_summary_24_25_1H = Coby_White_shot_location_avg_24_25_1H.copy()

# Calculate percentile rank for each stat column
for col in stat_columns_percentiles:
    Coby_White_shot_location_summary_24_25_1H[f'{col}_percentile'] = Coby_White_shot_location_summary_24_25_1H[col].apply(
        lambda x: get_percentile(x, shot_location_avg_24_25_1H_clean[col])
    )

# Remove unnecessary columns
essential_columns = ['PLAYER_ID', 'FULL_NAME', 'GROUP_SET', 'GROUP_VALUE']
percentile_columns = [f'{col}_percentile' for col in stat_columns_percentiles]
final_columns = essential_columns + stat_columns_percentiles + percentile_columns

Coby_White_shot_location_summary_24_25_1H = Coby_White_shot_location_summary_24_25_1H[final_columns]
Coby_White_shot_location_summary_24_25_1H.T

```

```

In [ ]: # Function to calculate percentrank for Coby's stats against all other players in 2024-25 season post all-star
def get_percentile(value, series):
    return percentileofscore(series, value, kind='rank')

Coby_White_shot_location_summary_24_25_2H = Coby_White_shot_location_avg_24_25_2H.copy()

# Calculate percentile rank for each stat column
for col in stat_columns_percentiles:
    Coby_White_shot_location_summary_24_25_2H[f'{col}_percentile'] = Coby_White_shot_location_summary_24_25_2H[col].apply(
        lambda x: get_percentile(x, shot_location_avg_24_25_2H_clean[col])
    )

```

```

# Remove unnecessary columns
essential_columns = ['PLAYER_ID', 'FULL_NAME', 'GROUP_SET', 'GROUP_VALUE']
percentile_columns = [f"{col}_percentile" for col in stat_columns_percentiles]
final_columns = essential_columns + stat_columns_percentiles + percentile_columns

Coby_White_shot_location_summary_24_25_2H = Coby_White_shot_location_summary_24_25_2H[final_columns]
Coby_White_shot_location_summary_24_25_2H.T

```

```

In [ ]: # Function to calculate percentrank for Coby's stats against all other players in 2023-24 season
def get_percentile(value, series):
    return percentileofscore(series, value, kind='rank')

Coby_White_shot_location_summary_23_24 = Coby_White_shot_location_avg_23_24.copy()

# Calculate percentile rank for each stat column
for col in stat_columns_percentiles:
    Coby_White_shot_location_summary_23_24[f'{col}_percentile'] = Coby_White_shot_location_summary_23_24[col].apply(
        lambda x: get_percentile(x, shot_location_avg_23_24_clean[col])
    )

# Remove unnecessary columns
essential_columns = ['PLAYER_ID', 'FULL_NAME', 'GROUP_SET', 'GROUP_VALUE']
percentile_columns = [f"{col}_percentile" for col in stat_columns_percentiles]
final_columns = essential_columns + stat_columns_percentiles + percentile_columns

Coby_White_shot_location_summary_23_24 = Coby_White_shot_location_summary_23_24[final_columns]
Coby_White_shot_location_summary_23_24.T

```

```

In [ ]: # Ensure all stat_columns are numeric
shot_location_avg_23_24.loc[:, stat_columns] = shot_location_avg_23_24[stat_columns].apply(pd.to_numeric, errors='coerce')
Coby_White_shot_location_avg_23_24.loc[:, stat_columns] = Coby_White_shot_location_avg_23_24[stat_columns].apply(pd.to_numeric, errors='coerce')

```

```

In [ ]: print(Coby_White_shot_location_avg_23_24[stat_columns].isnull().sum())
print(shot_location_avg_23_24[stat_columns].isnull().sum())

```

```

In [ ]: for col in stat_columns:
    bad_vals = shot_location_avg_23_24[pd.to_numeric(shot_location_avg_23_24[col], errors='coerce').isna()][col]
    if not bad_vals.empty:
        print(f"Bad values in {col}:")
        print(bad_vals.unique())

```

```

In [ ]: # Ensure all stat_columns are numeric
shot_location_avg_23_24_clean.loc[:, stat_columns] = shot_location_avg_23_24_clean[stat_columns].apply(pd.to_numeric, errors='coerce')
print(shot_location_avg_23_24_clean[stat_columns].isnull().sum())

```